

## **Course Grade Calculator**

# **SOFTWARE ARCHITECTURE AND DESIGN DOCUMENT**

# Table of Contents

---

## **1. Introduction**

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms and Abbreviations
- 1.4 References

## **2. Architectural Representation**

## **3. Architectural Goals and Constraints**

## **4. Use-Case View**

- 4.1 Architecturally-Significant Use Cases

## **5. Logical View**

- 5.1 Architecture Overview — Package and Subsystem Layering

## **6. Process View**

- 6.1 Processes
- 6.2 Process to Design Elements
- 6.3 Process Model to Design Model Dependencies
- 6.4 Processes to the Implementation

## **7. Deployment View**

- 7.1 Student Device
- 7.2 Web Application Hosting
- 7.3 Application/API Server
- 7.4 Database / Persistent Storage
- 7.5 External Services

## **8. Size and Performance**

## **9. Quality**

## **Architecture Summary**

## **Implementation Rule for Claude**

# 1. Introduction

---

## 1.1 Purpose

This document defines the software architecture of the Course Grade Calculator, a web-based academic calculation system designed to help university students calculate course results and understand their academic performance.

The purpose of this document is to establish the system structure before implementation begins. It provides a common architectural blueprint for developers and other stakeholders, and defines the major components, responsibilities, interactions, data flow, deployment structure, performance expectations, and quality requirements.

The system automates academic calculations that are commonly performed manually by students. Instead of calculating weighted assessment marks, final course marks, letter grades, grade points, GPA, or CGPA by hand, the student provides the required academic information and the system performs the calculations automatically.

The architecture separates presentation concerns from calculation and data-management concerns, so that changes to the user interface do not unnecessarily affect the calculation engine.

## 1.2 Scope

The scope of the Course Grade Calculator includes the following major areas.

### Course Management

The system will allow users to:

- Add a course.
- Enter course code and course name.
- Enter course credit.
- Edit course information.
- Remove a course.
- View all added courses.

### Assessment Management

A course may contain multiple assessment components such as:

- Attendance
- Assignment
- Quiz
- Class Test

- Midterm
- Final Examination
- Presentation
- Project
- Laboratory
- Other custom assessment components

The system will allow the user to define the score and weight of each assessment.

### Grade Calculation

The system will:

- Validate assessment marks.
- Validate assessment weights.
- Calculate weighted contributions.
- Calculate total course marks.
- Determine the corresponding letter grade.
- Determine the corresponding grade point.

### GPA Calculation

For multiple courses, the system calculates semester GPA using course credits and grade points. The conceptual formula is:

$$\text{GPA} = \Sigma(\text{Course Credit} \times \text{Grade Point}) / \Sigma(\text{Course Credit})$$

### CGPA Calculation

Where previous semester information is available, the system may calculate cumulative GPA using the total quality points and total credits across semesters.

$$\text{CGPA} = \text{Total Quality Points} / \text{Total Completed Credits}$$

### What-If Calculation

The system allows students to change expected assessment marks and immediately observe how the final course result changes.

### Result Presentation

The system will provide:

- Total marks

- Percentage
- Letter grade
- Grade point
- Course-level performance
- GPA
- CGPA where applicable

## Validation

The system will prevent invalid academic input such as:

- Negative marks
- Marks greater than the maximum allowed
- Invalid credit values
- Invalid assessment weights
- Empty required fields
- Assessment weights exceeding the allowed total

## Persistence

The architecture supports persistence of course and calculation data where required. The implementation may use browser storage for a lightweight deployment, or a backend database when user accounts and cross-device persistence are required.

## Responsive Access

The application will support:

- Desktop
- Laptop
- Tablet
- Mobile devices

## 1.3 Definitions, Acronyms and Abbreviations

| Term | Definition                        |
|------|-----------------------------------|
| GPA  | Grade Point Average               |
| CGPA | Cumulative Grade Point Average    |
| UI   | User Interface                    |
| UX   | User Experience                   |
| API  | Application Programming Interface |

| Term           | Definition                                                 |
|----------------|------------------------------------------------------------|
| REST           | Representational State Transfer                            |
| CRUD           | Create, Read, Update, Delete                               |
| SPA            | Single Page Application                                    |
| DB             | Database                                                   |
| BVA            | Boundary Value Analysis                                    |
| QA             | Quality Assurance                                          |
| HTTPS          | Hypertext Transfer Protocol Secure                         |
| Client         | User's browser/device                                      |
| Server         | Backend application server                                 |
| Assessment     | A graded component of a course                             |
| Weight         | Percentage contribution of an assessment                   |
| Weighted Score | Score contribution calculated using an assessment's weight |
| Grade Point    | Numerical representation of a letter grade                 |
| Course Credit  | Credit value assigned to a university course               |

## 1.4 References

The architecture is based on:

- The provided AI Paper Summarizer Software Architecture Document, used as the structural and documentation-style reference for this project.
- The existing Course Grade Calculator application concept and functional overview.
- Standard software architecture documentation practices.
- Standard layered/client-server architecture principles.
- University grading and GPA calculation rules defined by the target institution.

*Important: The grading scale must be configurable. The architecture must not hard-code one university's grading policy unless that policy is explicitly selected as a project requirement.*

## 2. Architectural Representation

---

The Course Grade Calculator follows a modular client-server architecture. For a complete production-ready version, the architecture is divided into five layers, with external services connected through the application/service layer:

```
graph TD; A[Presentation Layer] --> B[Application / API Layer]; B --> C[Business Logic Layer]; C --> D[Data Access Layer]; D --> E[Database];
```

Presentation Layer  
↓  
Application / API Layer  
↓  
Business Logic Layer  
↓  
Data Access Layer  
↓  
Database

## High-Level Architecture

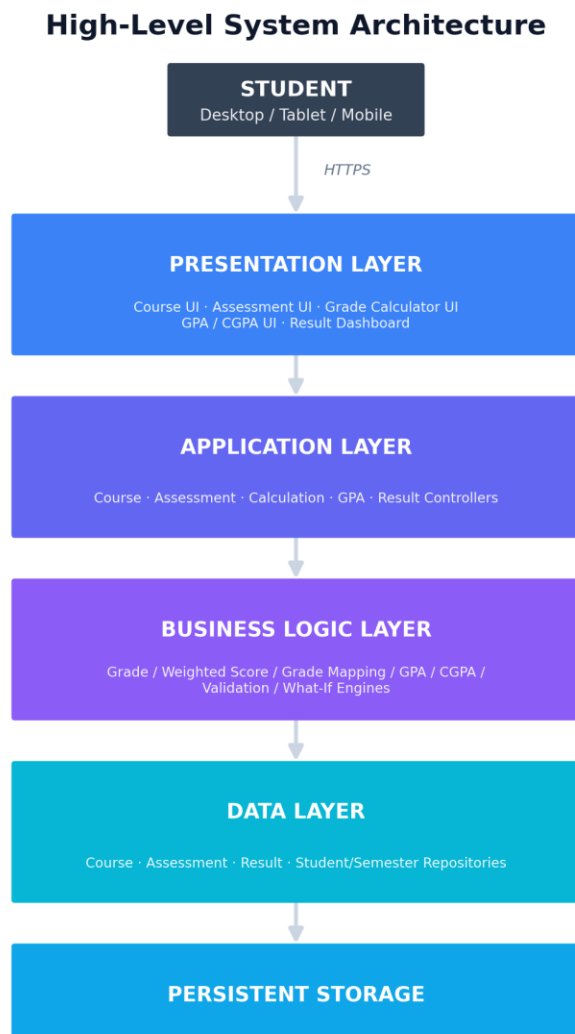


Figure 2.1 — High-level system architecture, from the student's device down through the presentation, application, business logic, and data layers to persistent storage.

## Architectural Style

The architecture uses:

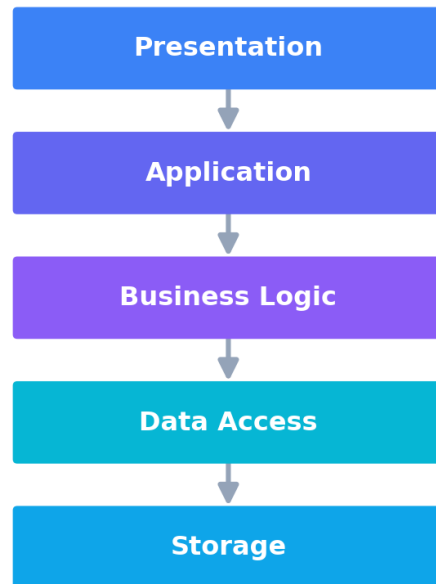
- Component-based frontend architecture.
- Layered application structure.
- Client-server separation.
- Service-oriented business modules.
- Repository/data-access abstraction.
- Configuration-driven grading rules.

The system should not be implemented as a collection of tightly coupled UI components where calculation formulas are directly embedded inside presentation components.



## Preferred Separation

### Preferred Architectural Separation



The calculation engine should remain independent of the UI framework and browser-specific presentation code.

## 3. Architectural Goals and Constraints

---

### 3.1 Architectural Goals

| Goal                   | Architectural Approach                |
|------------------------|---------------------------------------|
| Accuracy               | Centralized calculation engine        |
| Maintainability        | Modular layered architecture          |
| Usability              | Simple responsive interface           |
| Performance            | Lightweight calculation operations    |
| Extensibility          | Independent calculation services      |
| Reliability            | Validation and deterministic formulas |
| Testability            | Separation of UI and business logic   |
| Scalability            | Stateless application/API design      |
| Configurability        | Externalized grading rules            |
| Security               | HTTPS, validation, secure storage     |
| Accessibility          | Keyboard-friendly and readable UI     |
| Deployment Flexibility | Separate frontend/backend concerns    |

### 3.2 Functional Requirements

The system shall:

- Allow users to create course records.
- Allow users to enter course credits.
- Allow users to add assessment components.
- Allow users to enter assessment marks.
- Allow users to specify assessment weights.
- Validate marks and weights.
- Calculate weighted assessment scores.
- Calculate total course marks.
- Determine letter grade.
- Determine grade point.
- Calculate GPA.
- Calculate CGPA when semester history is available.
- Support modification of input values.

- Recalculate results after changes.
- Support what-if scenarios.
- Display results clearly.
- Allow resetting or removing course data.
- Preserve data when persistence is enabled.
- Support responsive access across devices.

### 3.3 Non-Functional Requirements

#### Performance

Normal calculations should complete almost immediately after valid input is provided.

#### Availability

The application should be accessible whenever the hosting platform is available.

#### Maintainability

Business rules should be separated from UI components.

#### Reliability

The same valid input must always produce the same calculated result.

#### Security

User-provided data must be validated before processing or persistence.

#### Usability

A student should be able to calculate a course grade without technical knowledge.

#### Scalability

The architecture should allow the addition of user accounts, semester history, analytics, and administrative grading configuration without redesigning the complete system.

### 3.4 Technical Constraints

- The system is primarily a web application.
- Calculations must be deterministic.
- Grading rules may vary between universities.
- The application must work on modern browsers.
- The interface must be responsive.
- The calculation engine must not depend on UI-specific code.

- Persistent storage must not expose sensitive information unnecessarily.
- The system should remain lightweight for users who only need calculation functionality.
- Any external database or backend service must be optional from an architectural perspective where possible.

### 3.5 Key Architectural Decisions

#### Decision 1 — Centralized Calculation Engine

All grade-related mathematical operations are handled by a dedicated calculation module.

#### Decision 2 — Configurable Grading Scale

Grade boundaries are represented as configuration rather than scattered throughout source code. Example:

```
Grade Configuration
-----
A+ → 4.00
A  → 3.75
A- → 3.50
B+ → 3.25
B  → 3.00
...
```

The exact scale will be defined according to the target university.

#### Decision 3 — Separation of UI and Business Logic

UI components collect and display information. They do not own academic calculation rules.

#### Decision 4 — Validation Before Calculation

Invalid data must be rejected before entering the calculation engine.

#### Decision 5 — Repository Abstraction

Persistence should be accessed through repository interfaces so that browser storage can later be replaced by a backend database without rewriting business logic.

## 4. Use-Case View

---

The Use-Case View describes how users interact with the Course Grade Calculator.

### Primary Actor

#### Student

The student is the main user. The student can:

- Enter course information.
- Enter assessment marks.
- Calculate course grade.
- Modify marks.
- Perform what-if calculations.
- Add multiple courses.
- Calculate GPA.
- Calculate CGPA.
- View academic results.
- Reset or remove data.

### Optional Secondary Actors

#### System Administrator

If a backend-enabled version is implemented, an administrator may:

- Configure grading scales.
- Manage university/department settings.
- Manage course templates.
- Monitor system usage.

#### Storage System

Stores course and result information where persistence is enabled.

### Use-Case Diagram

## Use-Case Diagram — Course Grade Calculator

Course Grade Calculator System

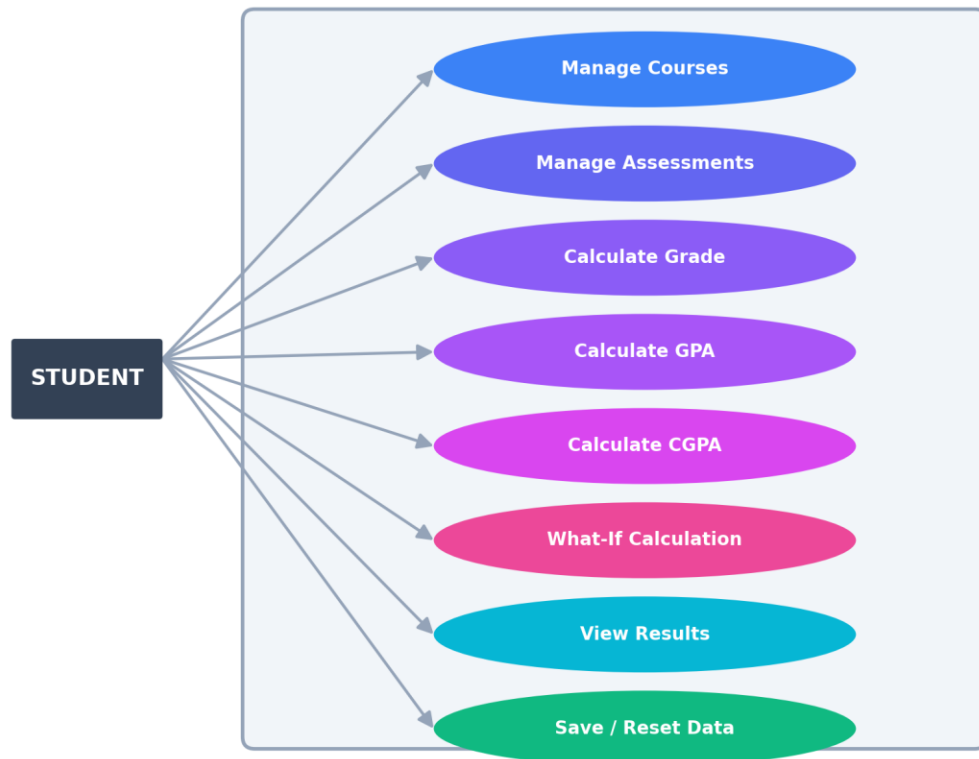


Figure 4.1 — Use-case diagram showing the student actor and the system's primary use cases.

## 4.1 Architecturally-Significant Use Cases

---

### UC-01: Calculate Course Grade

#### *Actor*

Student

#### *Preconditions*

- A course exists.
- Assessment information is available.
- Assessment marks are within valid ranges.
- Assessment weights are valid.

#### *Main Flow*

- 1. Student enters course information.
- 2. Student enters assessment components.
- 3. Student enters marks.
- 4. Student enters or selects assessment weights.
- 5. Validation engine checks the input.
- 6. Calculation engine calculates weighted contributions.
- 7. Total course score is calculated.
- 8. Grade mapping engine determines the letter grade.
- 9. Grade point is determined.
- 10. Result is returned to the UI.

#### *Postcondition*

The final course result is displayed.

### UC-02: Calculate GPA

#### *Main Flow*

- 1. Student adds multiple courses.
- 2. Each course has a valid credit value.
- 3. Each course has a grade point.
- 4. GPA engine calculates quality points.
- 5. Total quality points are divided by total credits.
- 6. GPA is displayed.

## Process 3 — GPA Calculation

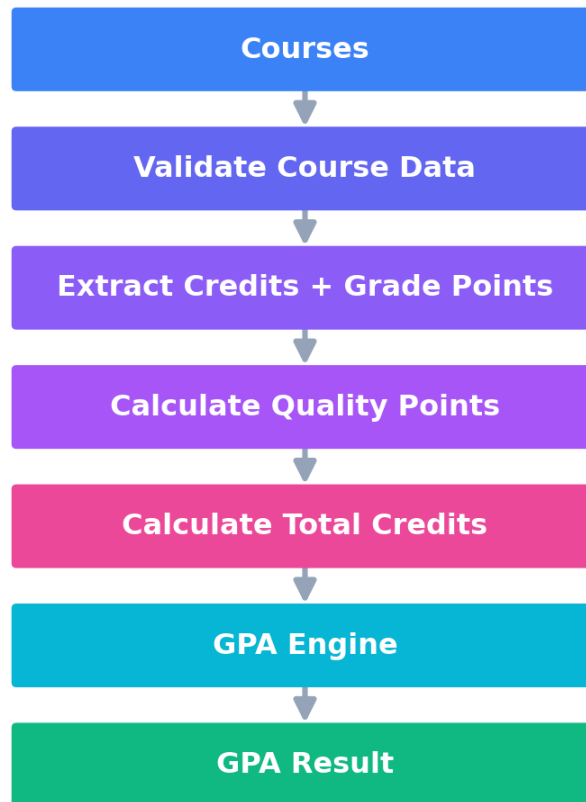


Figure 4.2 — GPA derivation from course credit and grade point.

### UC-03: Calculate CGPA

#### Main Flow

- 1. Student provides previous semester results.
- 2. Current semester results are added.
- 3. System calculates total quality points.
- 4. System calculates total completed credits.
- 5. CGPA engine performs cumulative calculation.
- 6. CGPA is displayed.

### UC-04: What-If Calculation

#### Main Flow

- 1. Student changes an assessment mark.



- 2. UI updates the calculation state.
- 3. Validation is performed.
- 4. Grade engine recalculates the result.
- 5. New grade/percentage is displayed.

This use case requires fast calculation and therefore has a direct impact on performance requirements.

## **UC-05: Manage Course**

The student can:

- Create course.
- Update course.
- Delete course.
- View course.
- Reset course.

This use case drives the course-management components and persistence layer.

## **UC-06: Validate Academic Input**

The system validates:

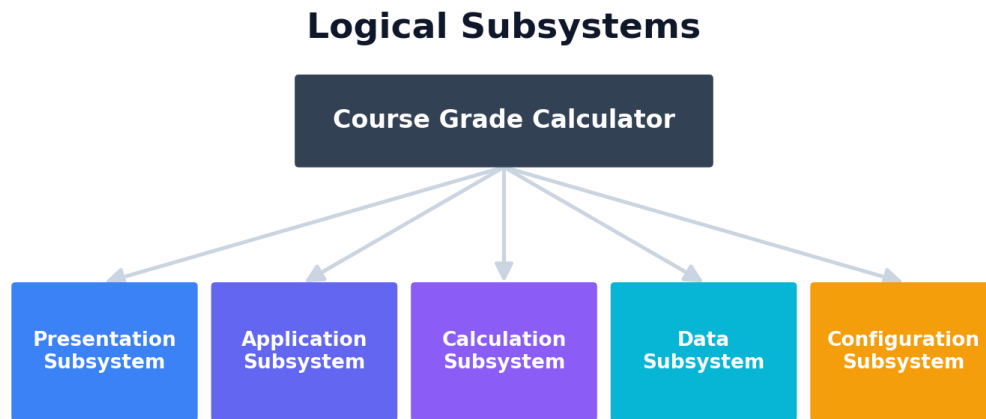
- Mark range.
- Weight range.
- Credit range.
- Required fields.
- Numeric format.
- Total assessment weight.

Invalid input must not reach the calculation engine.

## 5. Logical View

---

The Logical View describes the major software components and their responsibilities. The system is divided into five major logical subsystems.



*Figure 5.1 — The five logical subsystems of the Course Grade Calculator.*

## 5.1 Architecture Overview – Package and Subsystem Layering

---



Figure 5.2 — Detailed package and subsystem layering, from presentation through configuration.

### Layer 1 — Presentation Layer

Responsible for:

- Pages

- Forms
- Input controls
- Course cards
- Assessment tables
- Result cards
- GPA dashboard
- Validation messages
- Navigation
- Responsive layout

Example components:

```
pages/
├── Home
├── CourseCalculator
├── GPACalculator
├── CGPACalculator
└── Results

components/
├── CourseForm
├── AssessmentForm
├── AssessmentTable
├── GradeResult
├── GPAResult
├── CGPAResult
├── CourseCard
└── ValidationMessage
```

## Layer 2 — Application Layer

Coordinates user actions with business services.

```
CourseService
AssessmentService
GradeService
GPAService
CGPAService
StorageService
```

Responsibilities include:

- Receiving user actions.
- Calling appropriate business services.
- Managing application state.

- Coordinating persistence.
- Preparing results for presentation.

### Layer 3 — Business Logic Layer

This is the core of the system.

```
CalculationEngine
├── WeightedScoreCalculator
├── CourseGradeCalculator
├── GradePointCalculator
├── GPACalculator
├── CGPACalculator
├── WhatIfCalculator
└── ValidationEngine
```

#### Weighted Score Calculator

```
Weighted Score = Mark × Weight
```

where weight is represented as a fraction.

#### Course Grade Calculator

```
Final Course Score =  
Σ(Assessment Mark × Assessment Weight)
```

#### GPA Calculator

```
GPA =  
Σ(Credit × Grade Point)  
/  
Σ(Credit)
```

#### CGPA Calculator

```
CGPA =  
Total Quality Points  
/  
Total Completed Credits
```

## Layer 4 — Data Access Layer

The data layer hides the actual storage mechanism from business logic.

```
repositories/  
├── CourseRepository  
├── AssessmentRepository  
├── SemesterRepository  
└── ResultRepository
```

Possible implementations:

```
BrowserStorageRepository  
DatabaseRepository
```

This allows the application to move from local persistence to a remote database without rewriting the calculation engine.

## Layer 5 — Configuration Layer

The configuration layer contains:

```
GradingScale  
AssessmentRules  
ValidationRules  
ApplicationSettings
```

Example:

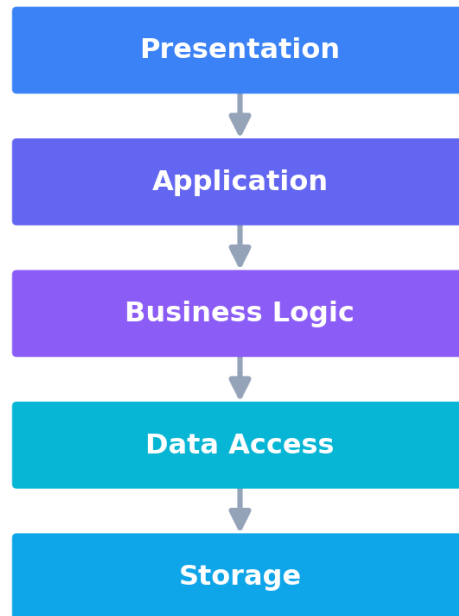
```
GradeRule  
{  
    minimumPercentage,  
    maximumPercentage,  
    letterGrade,  
    gradePoint  
}
```

This makes the system adaptable to different university grading policies.

## Logical Dependency Rule

Dependencies should flow downward:

## Logical Dependency Rule (Downward)



The calculation engine should not depend on React components, HTML, or browser-specific APIs.

## 6. Process View

---

The Process View explains how the system behaves during runtime. The main runtime processes are:

- 1. Application initialization.
- 2. Course creation.
- 3. Assessment entry.
- 4. Input validation.
- 5. Grade calculation.
- 6. GPA calculation.
- 7. CGPA calculation.
- 8. What-if recalculation.
- 9. Data persistence.
- 10. Result rendering.

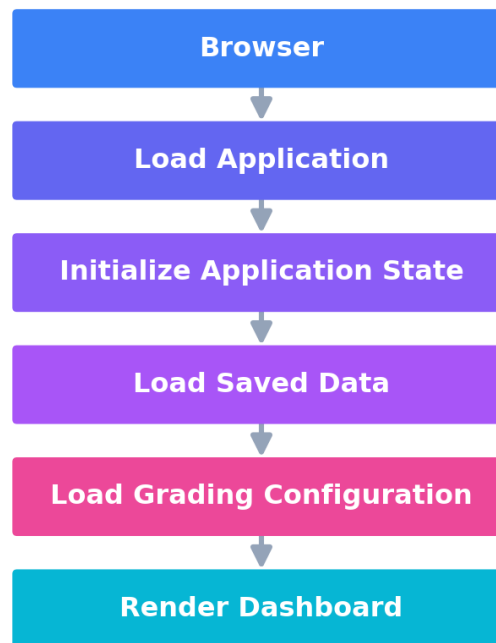


## 6.1 Processes

---

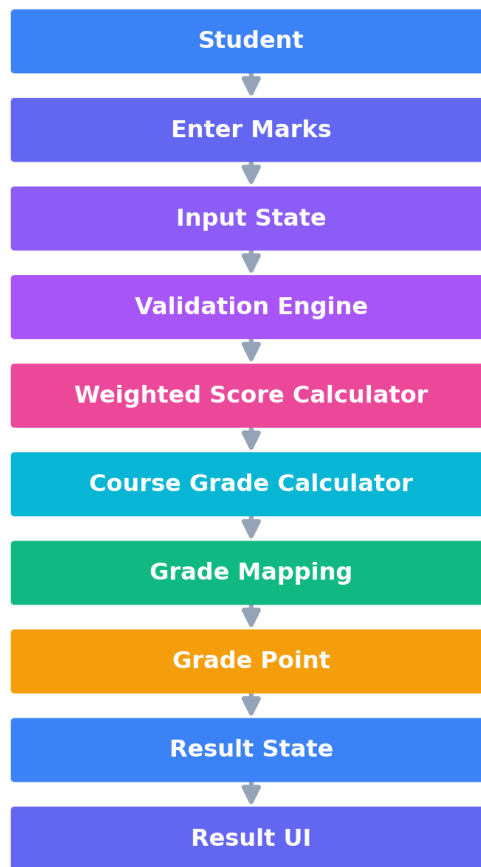
### Process 1 — Application Initialization

#### Process 1 — Application Initialization



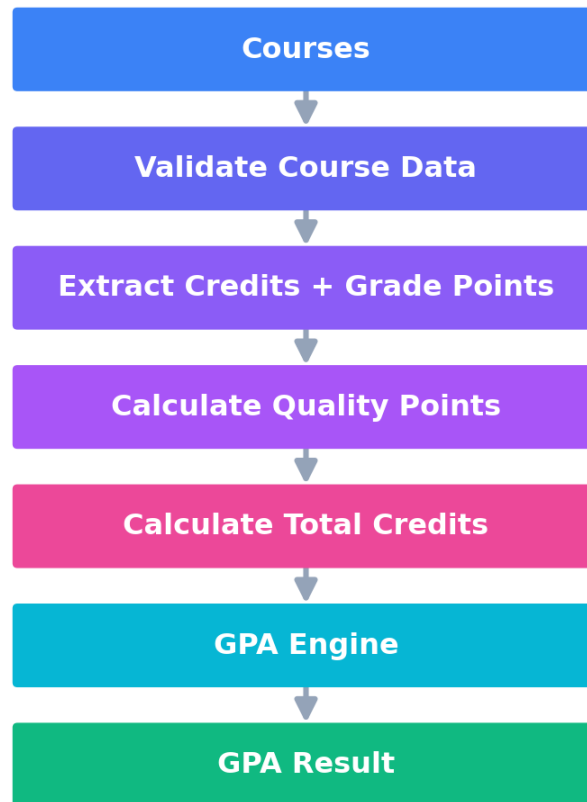
### Process 2 — Course Grade Calculation

## Process 2 — Course Grade Calculation



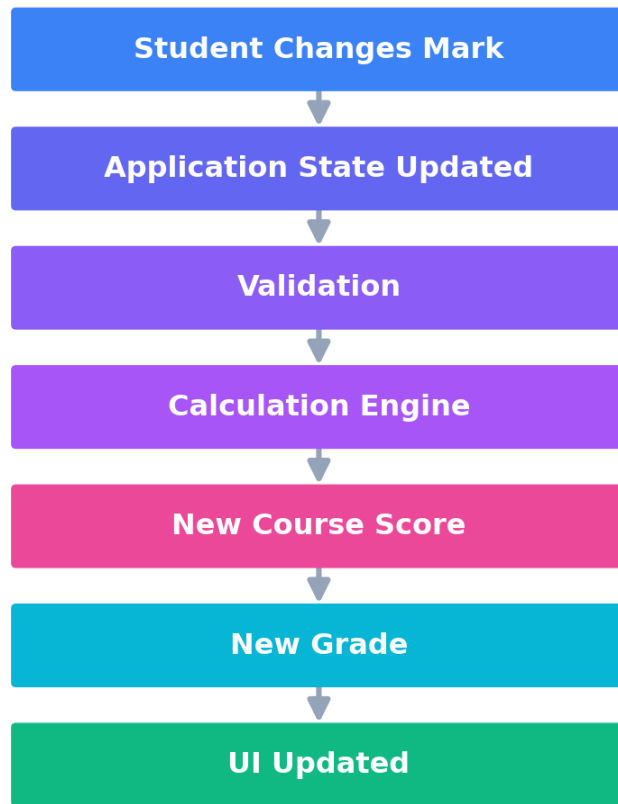
## Process 3 — GPA Calculation

## Process 3 — GPA Calculation



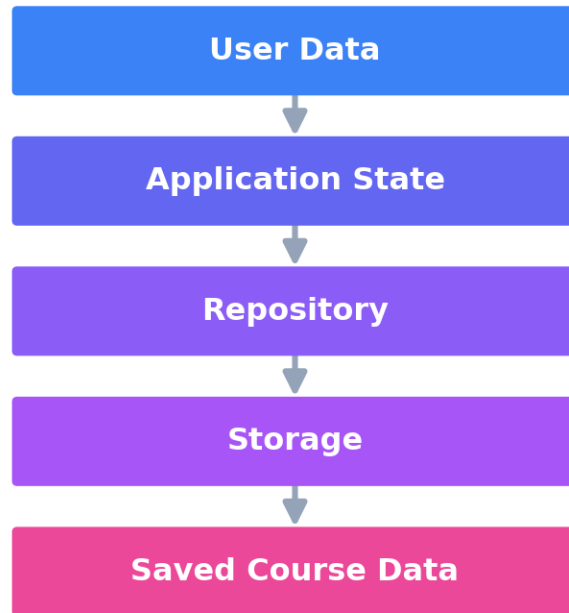
## Process 4 — What-If Calculation

## Process 4 — What-If Calculation



## Process 5 — Persistence

## Process 5 — Persistence



## 6.2 Process to Design Elements

---

| Runtime Process            | Design Elements                   |
|----------------------------|-----------------------------------|
| Application initialization | AppShell, StateManager            |
| Course creation            | CourseForm, CourseService         |
| Assessment entry           | AssessmentForm, AssessmentService |
| Validation                 | ValidationEngine                  |
| Weighted calculation       | WeightedScoreCalculator           |
| Course grade               | CourseGradeCalculator             |
| Grade mapping              | GradePointCalculator              |
| GPA                        | GPACalculator                     |
| CGPA                       | CGPACalculator                    |
| What-if calculation        | WhatIfCalculator                  |
| Persistence                | Repository + Storage              |
| Result display             | ResultComponent                   |

## 6.3 Process Model to Design Model Dependencies

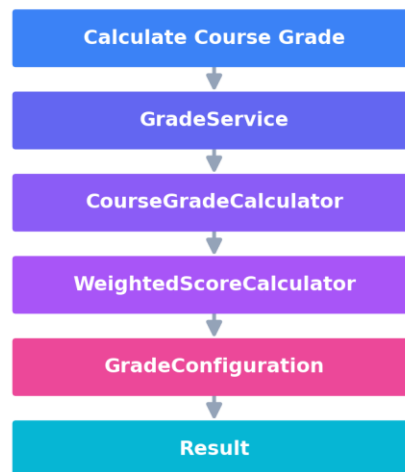
---

The process architecture depends on the logical architecture as follows:

```
Process
↓
Application Service
↓
Business Service
↓
Calculation Engine
↓
Repository
↓
Storage
```

Example:

### Process Model → Design Model Dependency (Example)



The process model therefore does not create independent calculation logic. It uses the reusable components defined in the Logical View.

## 6.4 Processes to the Implementation

---

The implementation should follow this conceptual structure:

```
src/
├── components/
├── pages/
├── services/
├── calculations/
│   ├── weightedScore.js
│   ├── courseGrade.js
│   ├── gradePoint.js
│   ├── gpa.js
│   ├── cgpa.js
│   └── whatIf.js
├── validation/
│   └── validationRules.js
├── repositories/
│   ├── courseRepository.js
│   ├── assessmentRepository.js
│   └── resultRepository.js
├── config/
│   └── gradingScale.js
├── state/
└── utils/
```

If a backend is used:

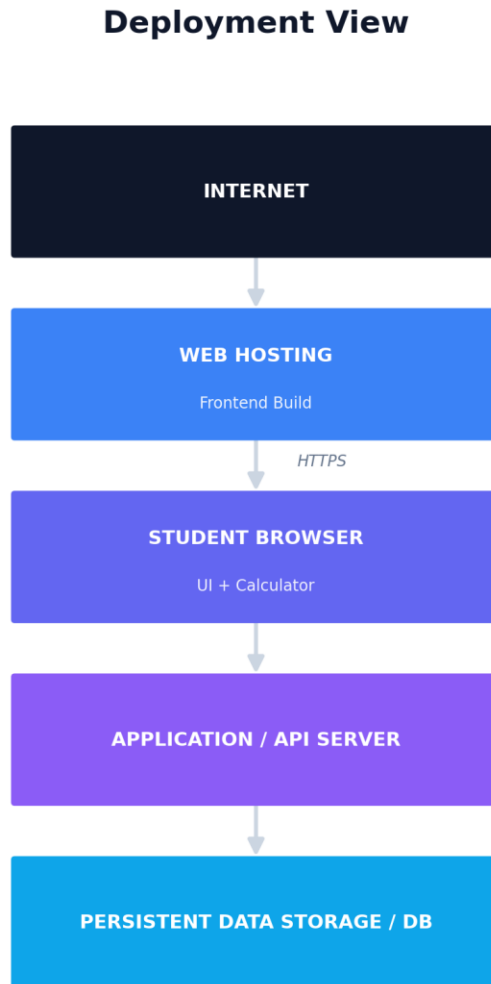
```
backend/
├── routes/
├── controllers/
├── services/
├── calculations/
├── repositories/
├── models/
├── validation/
├── config/
└── database/
```



## 7. Deployment View

---

The deployment architecture defines where the software components execute.



*Figure 7.1 — Deployment view from internet access through hosting, browser, API server, and storage.*

For a lightweight client-side deployment, the calculation engine may execute completely inside the browser. For a full multi-user version, persistent data and account-related operations can be moved to the backend.

## 7.1 Student Device

---

The student's device runs the presentation layer. Supported environments:

- Desktop browser
- Laptop browser
- Tablet browser
- Mobile browser

The device is responsible for:

- Rendering the UI.
- Receiving user input.
- Maintaining temporary application state.
- Executing lightweight calculations where appropriate.
- Displaying results.

No sensitive server-side secrets should be embedded in frontend code.

## 7.2 Web Application Hosting

---

The frontend application will be deployed to a web-hosting platform. The hosting layer is responsible for:

- Serving HTML.
- Serving JavaScript bundles.
- Serving CSS.
- Serving static assets.
- Providing HTTPS.
- Delivering the application to users.

The current reference application is available as a web application hosted under a Firebase web.app domain. The architecture therefore keeps the hosting layer independent from the calculation logic so that deployment remains flexible.

## 7.3 Application/API Server

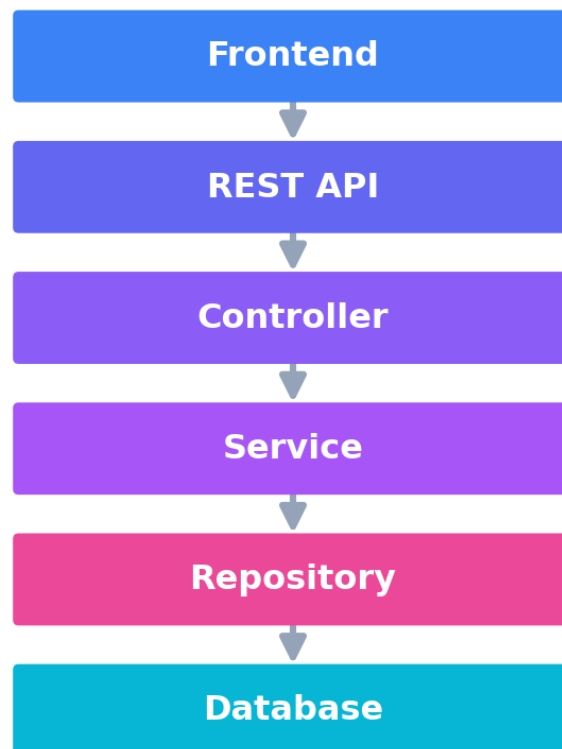
---

For the complete backend-enabled architecture, an application server handles:

- User-related operations.
- Course persistence.
- Semester history.
- Database communication.
- API requests.
- Server-side validation.
- Authentication if accounts are introduced.

Conceptually:

### Application/API Server Flow



The backend should remain stateless for normal API requests wherever possible.

## 7.4 Database / Persistent Storage

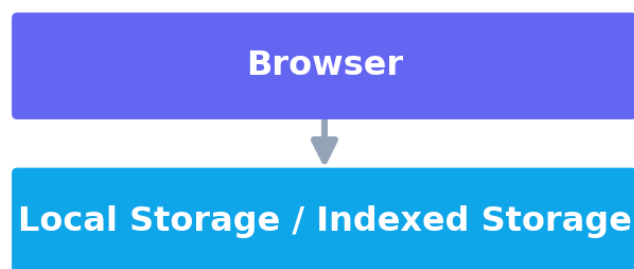
---

The architecture supports two persistence levels.

### Level 1 — Lightweight Version

Browser-based storage:

## Level 1 — Lightweight Version



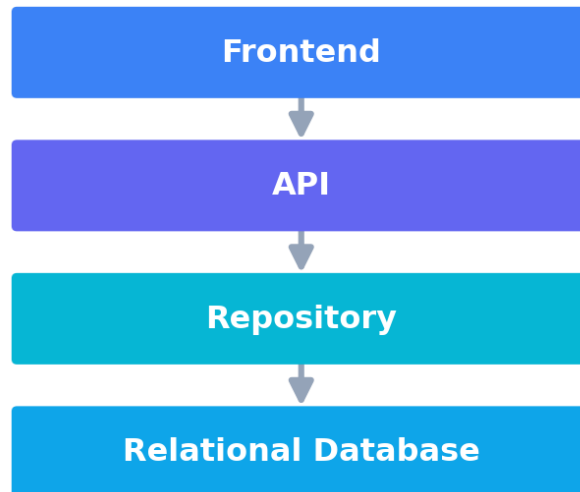
Suitable for:

- Personal calculations.
- No-login usage.
- Offline-friendly behavior.
- Small data volume.

### Level 2 — Full Multi-User Version

Remote database:

## Level 2 — Full Multi-User Version

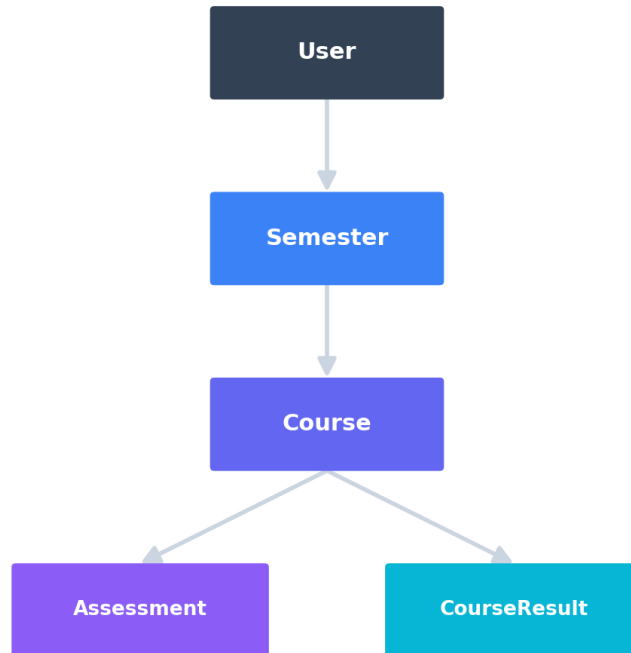


Potential entities include:

```
User  
Course  
Assessment  
Semester  
CourseResult  
GradingScale
```

### Conceptual Data Model

## Conceptual Data Model



*Figure 7.2 — Conceptual data model relating users, semesters, courses, assessments, and results.*

## 7.5 External Services

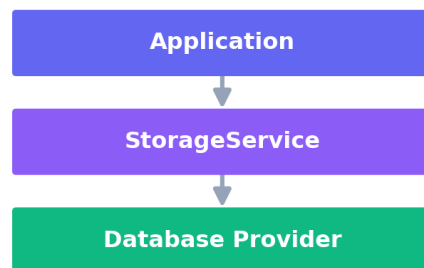
---

The system should minimize unnecessary external dependencies. Possible external services include:

- Hosting provider.
- Database provider.
- Authentication provider.
- Monitoring service.
- Analytics service.

External services should be isolated behind service interfaces wherever practical, for example:

### External Services — Isolated Behind Interfaces



rather than allowing every UI component to directly communicate with the database.



## 8. Size and Performance

---

### 8.1 Expected System Size

The Course Grade Calculator is expected to be a relatively lightweight application. The core calculation operations are computationally inexpensive because they primarily involve:

- Addition.
- Multiplication.
- Division.
- Comparisons.
- Sorting/filtering of small grade-rule collections.

Therefore, mathematical calculation is not expected to be the primary performance bottleneck.

### 8.2 Performance Targets

| Operation                   | Target                      |
|-----------------------------|-----------------------------|
| Initial application loading | Fast on normal broadband    |
| Course calculation          | Near-instantaneous          |
| What-if calculation         | Near-instantaneous          |
| GPA calculation             | Near-instantaneous          |
| CGPA calculation            | Near-instantaneous          |
| Local data retrieval        | Near-instantaneous          |
| Normal API request          | Low latency                 |
| Result rendering            | Immediate after calculation |

### 8.3 Scalability

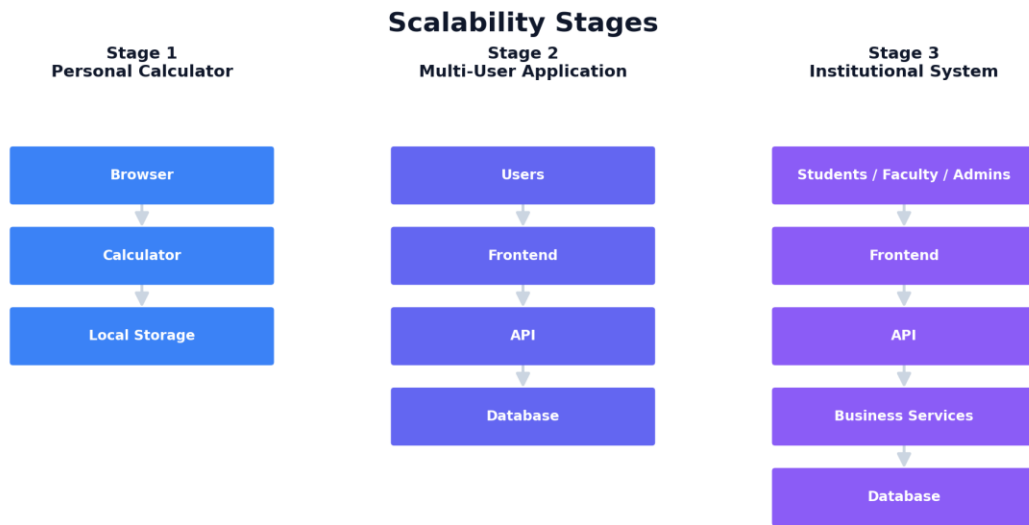


Figure 8.1 — Three scalability stages: personal calculator, multi-user application, and institutional system.

## 8.4 Performance Risks

Potential performance risks include:

- 1. Excessively large frontend bundles.
- 2. Unnecessary API calls.
- 3. Recalculating unchanged data.
- 4. Poor state management.
- 5. Excessive database queries.
- 6. Rendering large course histories unnecessarily.
- 7. Network dependency for calculations that could be performed locally.

## Mitigation

- Keep calculations local where possible.
- Use memoization for derived results where useful.
- Avoid unnecessary re-rendering.
- Lazy-load non-critical pages.
- Minimize API requests.
- Use indexed database queries for persistent data.
- Cache static grading configuration.

## 9. Quality

---

The architecture is evaluated against the major software quality attributes relevant to the Course Grade Calculator.

### 9.1 Accuracy

PRIORITY: **CRITICAL**

The most important quality attribute is calculation accuracy.

```
Same Input
  ↓
Same Calculation Rules
  ↓
Same Result
```

All formulas must be centralized and tested independently.

### 9.2 Performance

PRIORITY: **HIGH**

Grade calculations are computationally simple and should provide results immediately. The application should not require a network request for every basic mathematical operation when those calculations can safely execute locally.

### 9.3 Usability

PRIORITY: **HIGH**

The system is intended for students and should provide:

- Simple forms.
- Clear labels.
- Helpful validation messages.
- Visible calculation results.
- Minimal unnecessary steps.
- Responsive design.
- Understandable academic terminology.

### 9.4 Maintainability

PRIORITY: **HIGH**

Maintainability is achieved through:

- Separation of concerns.
- Modular components.
- Centralized formulas.
- Configurable grading rules.
- Repository abstraction.
- Reusable validation functions.

A grading-scale change should ideally require modification of configuration rather than modification of multiple UI components.

## 9.5 Reliability

PRIORITY: **HIGH**

The system should consistently produce correct results for valid input. Reliability is supported by:

- Deterministic calculations.
- Input validation.
- Unit tests.
- Boundary testing.
- Error handling.
- Safe persistence.

## 9.6 Security

PRIORITY: **MEDIUM/HIGH**

For the basic calculator, security requirements are relatively low because academic calculations do not inherently require sensitive information. For a multi-user version, security becomes more important.

Required controls include:

- HTTPS.
- Input validation.
- Authentication.
- Authorization.
- Secure password handling.
- Protected API endpoints.
- Database access control.
- Secure environment variables.
- No secret keys in frontend source code.

## 9.7 Scalability

PRIORITY: **MEDIUM**

The architecture should support increasing users and data without requiring a complete redesign. The layered structure allows:

```
Local Storage
  ↓
Remote Database
  ↓
Institutional Deployment
```

without changing the core calculation formulas.

## 9.8 Testability

PRIORITY: **HIGH**

The calculation engine should be independently testable. Example:

```
Input:
Mark = 80
Weight = 0.30

Expected:
Weighted Score = 24
```

The following should be tested separately:

- Weighted score calculation.
- Course total calculation.
- Grade mapping.
- GPA calculation.
- CGPA calculation.
- Validation.
- Boundary values.
- What-if calculations.

## 9.9 Modifiability

PRIORITY: **HIGH**

The system must allow future changes such as:

- New grading scales.
- New assessment types.

- New universities.
- New GPA rules.
- Semester history.
- User authentication.
- Academic analytics.
- Result export.
- PDF report generation.

These changes should be possible without rewriting the entire application.

## 9.10 Availability

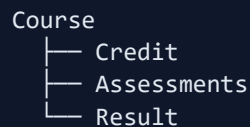
PRIORITY: **MEDIUM**

The application should be available whenever the hosting infrastructure is operational. A client-side calculation mode can provide an additional advantage because basic calculations can continue without requiring continuous backend communication.

## 9.11 Data Integrity

PRIORITY: **HIGH**

Course, assessment, credit, and result relationships must remain consistent. For example:



A course should not contain an invalid or orphaned assessment record.

## 9.12 Accessibility

PRIORITY: **HIGH**

The interface should support:

- Keyboard navigation.
- Readable typography.
- Appropriate form labels.
- Accessible buttons.
- Sufficient contrast.
- Clear error messages.
- Responsive layouts.

## 9.13 Architectural Trade-offs

| Decision                   | Benefit                          | Trade-off                       |
|----------------------------|----------------------------------|---------------------------------|
| Client-side calculation    | Very fast                        | Limited centralized control     |
| Local storage              | Simple and inexpensive           | Device-specific data            |
| Backend database           | Cross-device persistence         | More infrastructure             |
| Configurable grading scale | Flexible                         | More configuration complexity   |
| Layered architecture       | Maintainable                     | More initial structure          |
| Repository abstraction     | Testable and replaceable storage | Additional abstraction          |
| What-if calculation        | Very useful to students          | Requires frequent recalculation |
| Optional authentication    | Supports multi-user system       | Adds security complexity        |

## 9.14 Architectural Risks

| Risk                      | Impact                | Priority | Mitigation                                   |
|---------------------------|-----------------------|----------|----------------------------------------------|
| Incorrect formula         | Wrong academic result | Critical | Central calculation engine + tests           |
| Incorrect grading scale   | Wrong grade/GP        | Critical | Configurable grading rules                   |
| Invalid input             | Incorrect calculation | High     | Validation engine                            |
| Data loss                 | Lost course records   | Medium   | Persistent storage and backup                |
| Browser-specific behavior | UI problems           | Medium   | Cross-browser testing                        |
| Poor mobile layout        | Reduced usability     | Medium   | Responsive design                            |
| Backend failure           | Data unavailable      | Medium   | Local calculation fallback where appropriate |
| Contract mismatch         | Frontend/API errors   | Medium   | Formal API schema                            |
| Tight coupling            | Difficult maintenance | Medium   | Layered architecture                         |
| Security weakness         | Unauthorized access   | High     | Authentication and authorization             |

## 9.15 Recommended Future Architecture

The initial architecture is intentionally simple enough to implement efficiently, but it should support future expansion. Potential future features include:

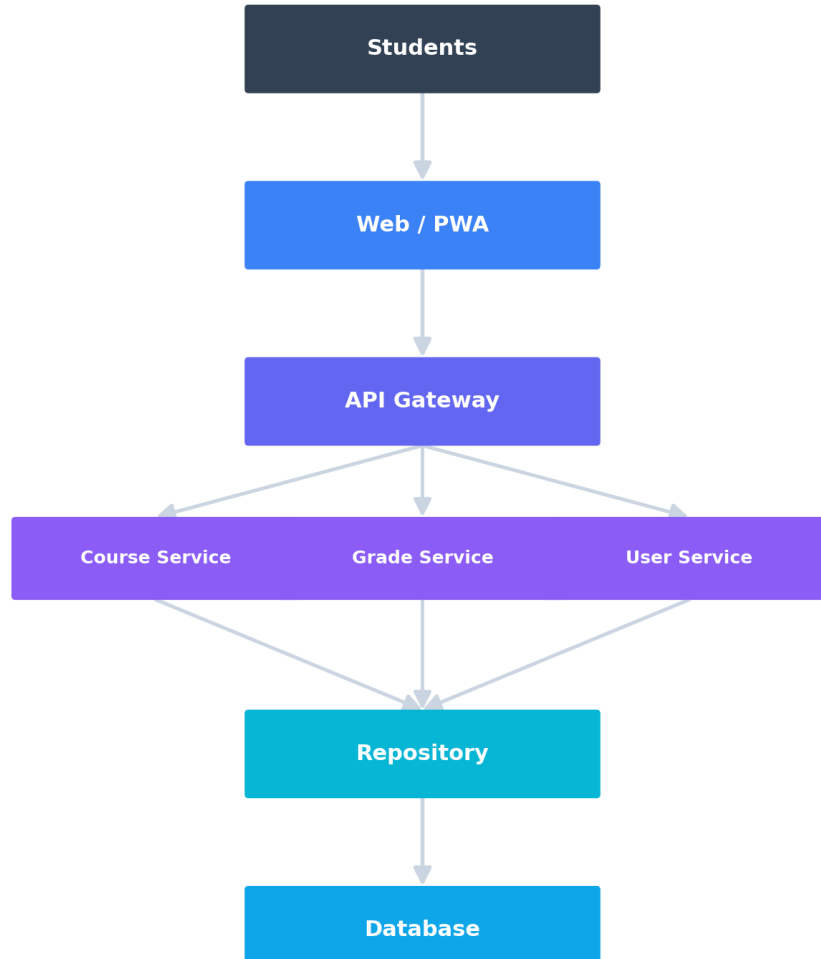
- 1. Student accounts.
- 2. Multiple semester management.
- 3. University-specific grading profiles.

- 4. Course templates.
- 5. GPA/CGPA history.
- 6. Academic performance charts.
- 7. Grade prediction.
- 8. What-if scenario comparison.
- 9. PDF result export.
- 10. CSV export.
- 11. Cloud synchronization.
- 12. Offline Progressive Web App support.
- 13. Administrative grading-scale management.
- 14. Institutional deployment.
- 15. Accessibility improvements.
- 16. Centralized monitoring.

A future institutional architecture may be:



## Recommended Future Architecture



*Figure 9.1 — Possible future evolution toward an institutional, service-oriented architecture. Not required for the first implementation.*

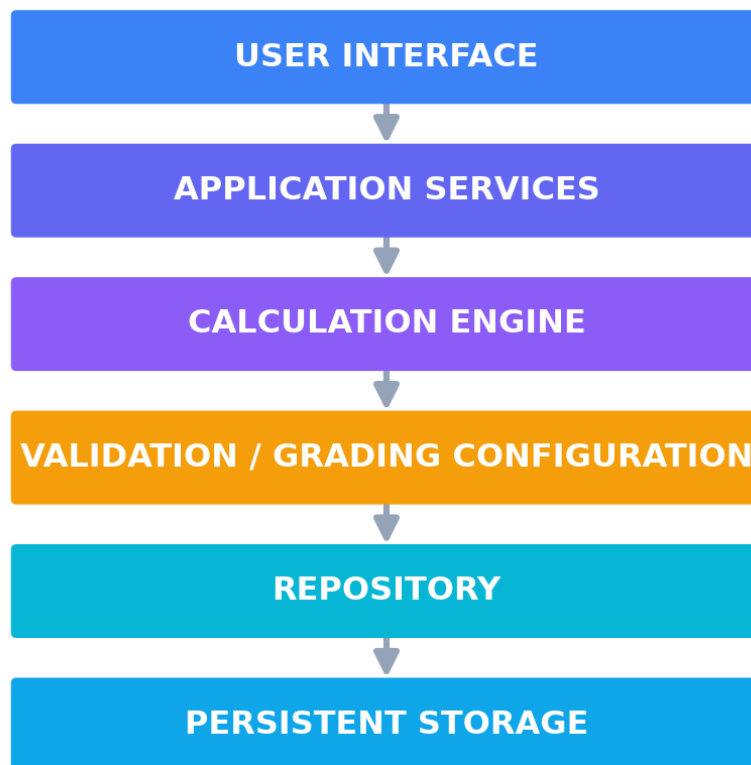
This future architecture is not required for the first implementation. It represents a possible evolution path.

## Architecture Summary

---

The Course Grade Calculator should be implemented as a modular, layered web application. The most important architectural principle is:

### Core Architectural Principle



*Figure — The core architectural principle: UI down through application services, calculation engine, validation/configuration, repository, and persistent storage.*

The calculation engine is the core of the system. It must remain independent from the frontend so that:

- UI changes do not affect formulas.
- Grading rules can be changed independently.
- GPA/CGPA logic can be tested independently.
- Future mobile applications can reuse the same calculation rules.
- A backend can be introduced without rewriting the core business logic.

The architecture provides a foundation for implementing the Course Grade Calculator incrementally while maintaining accuracy, usability, performance, maintainability, and extensibility.

## Implementation Rule for Claude

---

When implementing this project from this document:

- 1. Follow the architecture before writing features.
- 2. Do not place calculation formulas directly inside UI components.
- 3. Keep grading rules configurable.
- 4. Keep validation separate from presentation.
- 5. Keep storage access behind repository/service abstractions.
- 6. Reuse calculation functions rather than duplicating formulas.
- 7. Do not introduce unnecessary microservices.
- 8. Do not add technologies that are not required by the architecture.
- 9. Build the application incrementally by subsystem.
- 10. Write tests for every calculation rule before considering the calculation module complete.
- 11. Treat accuracy as the highest-priority quality attribute.
- 12. Keep the interface responsive.
- 13. Keep the architecture simple for the initial release.
- 14. Clearly distinguish implemented functionality from future functionality.
- 15. Any change to the architecture should be documented before implementation.

## Recommended Implementation Order

## Recommended Implementation Order

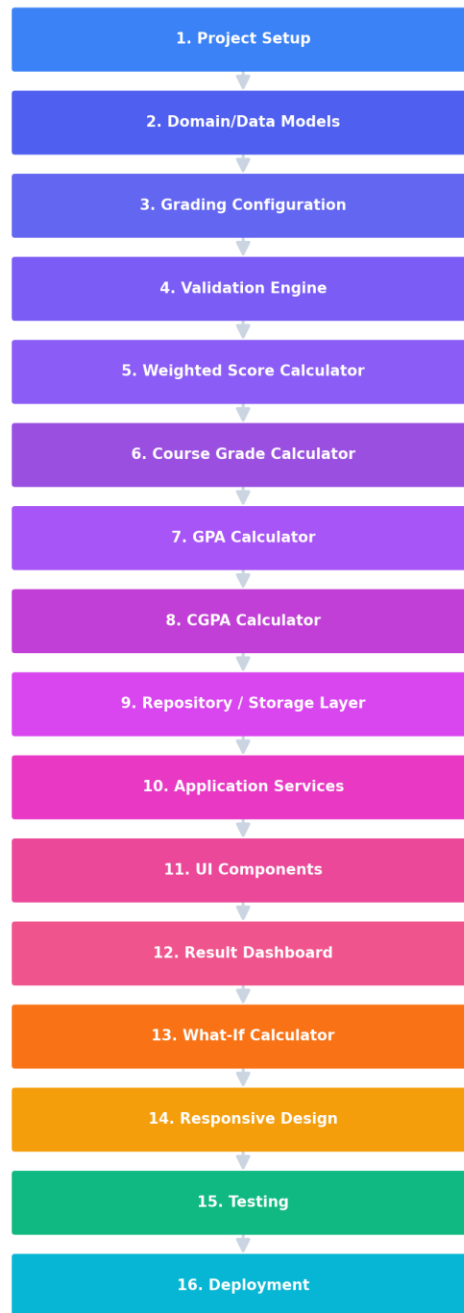


Figure — Recommended sixteen-step implementation order, from project setup through deployment.

## Final Architectural Principle

*The Course Grade Calculator is a calculation-centered academic application. Therefore, correctness, separation of business logic, configurable grading rules, and testability must take priority over unnecessary architectural complexity.*